Predicting bina.az Sale Prices and Price Tiers

From-scratch decision tree and Pegasos SVM

Esmira Mammadli Gunash Mammadova Ali Muradov Punhan Mursalov

MLE-AI-201 Machine Learning, AI Academy — Cohort II

1. Problem and dataset

- Data: Kaggle *binaaz-sale-project*, 100,775 scraped rows, 51 columns, prices in AZN.
- **Task A**: regress the listing price.
- **Task B**: classify *premium* vs. *standard*; premium = price strictly above the **training median** (210,000 AZN).
- Models written from scratch in NumPy: decision tree, Pegasos SVM (+ random Fourier features).
- Compared with scikit-learn models with **matched** settings; test set scored **once**.

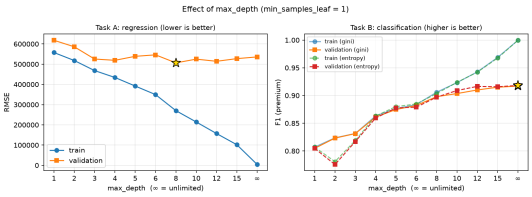
Raw rows	Distinct listings	Cleaned	Features
100,775	64,454	**64,350**	171

2. Cleaning, features and the leakage we removed

- One row per listing: keep the latest scrape (the same listing was in train *and* test before).
- Plausibility filters (104 rows): price outside 5,000–20M AZN, area outside 8–10^6 m^2 (e.g. an 11 AZN flat).
- Areas in *sot* converted to m^2; floor and building floors parsed; medians imputed; one-hot categories.
- **Leakage columns removed:** `unit_price`, `total_price` (derived from the target), plus IDs, URLs, free text, scrape time, view counts, currency.
- Split 70/15/15, seed 42, stratified by tier: 45,045 / 9,652 / 9,653. Scaler fitted on train only.

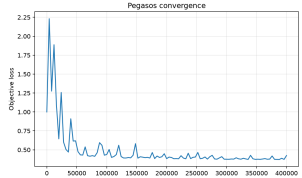
3. Our decision tree

- Impurity: Gini, entropy (Task B); MSE = variance (Task A).
- Split search: sort each feature once, scan midpoints with cumulative counts (or cumulative sums for MSE).
- Pick the split with the largest weighted impurity decrease; recurse.
- Hyperparameters: `max_depth`, `min_samples_leaf` (also `min_samples_split`, `min_impurity_decrease`).
- Chosen on validation: Task A depth 8, leaf 10 (125 leaves); Task B entropy, unlimited depth (3,068 leaves).



4. Our Pegasos SVM

- Hinge primal: $J = \frac{\lambda}{2}\|w\|^2 + \frac{1}{n}\sum_i \max(0, 1 - y_i(w^\top x_i + b))$.
- Update, $\eta_t = 1/(\lambda t)$, one random sample: $w \leftarrow (1 - \eta_t \lambda) w + \eta_t y_i x_i \, [y_i f(x_i) < 1]$, $b \leftarrow b + \eta_t y_i$.
- Non-linear extension: random Fourier features $\phi(x) = \sqrt{2/D}\cos(\omega^\top x + b)$, $\omega \sim \mathcal{N}(0, 2\gamma I)$; then linear Pegasos.
- Joint (λ, γ) grid on validation. Chosen: $D = 1024$, $\gamma = 0.01$, $\lambda = 3 \cdot 10^{-5}$, 400,000 updates.
- Final model: $\|w\| = 39.6$, margin width $2/\|w\| = 0.0505$, 17,528 training points on or inside the margin; val F1 0.845.



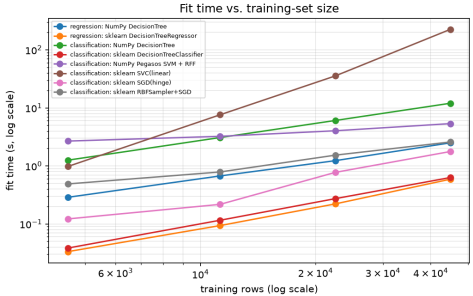
5. Experimental setup

- Metrics: RMSE, MAE, R^2 (and on log price); accuracy, precision, recall, F1, ROC-AUC, PR-AUC. All NumPy.
- Validation picks hyperparameters; **test scored once**; 5-fold CV (stratified for Task B) on train+validation.
- Seed 42 everywhere; `python -m src.run_all` reproduces every number (deterministic).
- Baselines (matched): sklearn tree with the same settings; linear SVC with $C = 1/(\lambda n)$; SGD hinge with $\alpha = \lambda$; RBFSampler+SGD for the RFF model.
- Fit times are per-fold wall-clock on one laptop: read them as ratios.

6. Results (test set)

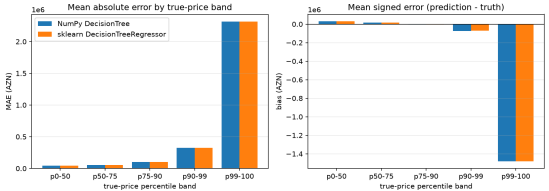
Model	A: RMSE	B: F1
Our tree	358,672	0.919
sklearn tree	358,999	0.918
Our SVM (RFF)	—	0.838
sklearn SVC (linear)	—	0.839
sklearn RBFSampler+SGD	—	0.833

- Tree matches sklearn (CV RMSE $438{,}746 \pm 29{,}122$; CV F1 0.915 ± 0.003).
- SVM = SVC in F1 (CV 0.843 both), ROC-AUC 0.924 vs 0.923; fit 5.3 s vs 245 s per fold ($\approx 46\times$ faster).
- Tree has higher F1, SVM higher ROC-AUC (0.924 vs 0.920).



7. Where the models fail, and limitations

- Top 1% of prices (99 listings): bias −1.48M AZN, MAE 2.31M. Cheap listings are over-predicted (+29,928).
- Sparse districts (fewer than 200 train rows): median relative error 0.355 vs 0.177 elsewhere.
- SVM: precision 0.768 vs recall 0.922, it over-predicts premium. A tuned threshold may help (*hypothesis, not tested*).
- Asking prices, not sale prices; about 700 possible re-posted ads kept; one seed, one split.
- Our NumPy tree is 5–30× slower than sklearn's (CV fit 4.5 s vs 1.0 s, 20.5 s vs 0.7 s).



8. Who did what, and what next

- **Punhan Mursalov**: data loading and cleaning, leakage columns, features, split, EDA, README; PCA / k-means / DBSCAN bonus.
- **Gunash Mammadova**: decision tree (impurity, split search, importances), tree tests and experiments; random forest bonus.
- **Ali Muradov**: Pegasos SVM, random Fourier features, SVM experiments and tests; AdaBoost bonus.
- **Esmira Mammadli**: metrics, CV and timing, sklearn baselines, benchmark, error analysis, `run_all`, pinned requirements, report merge; ridge/SVR bonus.
- AI use (small code corrections, help with the LaTeX files) is disclosed in the report and in `contribution_report.md`.

Next: tune the SVM threshold; model log price for luxury listings; de-duplicate re-posted ads; more seeds/splits.

Bonus models (our NumPy code; sklearn only for comparison)

Model	Metric	Ours	sklearn
Random forest (30 trees)	Task B F1	0.936	0.936
Random forest	Task A RMSE	318,706	343,257
AdaBoost (depth 3, 100 rounds)	Task B F1	0.904	0.900
Ridge / ε-SVR	Task A RMSE	583,307 / 583,895	—
Our single tree (reference)	Task A RMSE	358,672	—

- Linear models are far worse than the tree: Task A is strongly non-linear.
- Clustering: 20 PCs explain only 22.1% of variance; K-means / DBSCAN ARI against tier at most 0.072.